

P1180R0

Response to P1156

Published Proposal, 2018-09-18

This version:

<http://wg21.link/p1180r0>

Author:

[Richard Smith](#) (Google)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper discusses some issues with the preamble as described in the merged modules proposal (P1103R0), particularly in light of P1156 and implementation concerns raised since P1103R0 was discussed.

Table of Contents

1 Background

- 1.1 Version of P1156
- 1.2 What is the preamble for?

2 P1156R0 Section 3: Non-Modular Code

- 2.1 Proposal

3 P1156R0 Section 4: Preamble End

- 3.1 Do not allow preprocessor action at the end of the preamble
- 3.2 Defer macro import slightly more

3.3 Do not defer macro import

3.4 Proposal

4 P1156R0 Section 5: Module Partitions

§ 1. Background

§ 1.1. Version of P1156

This paper is based on a draft of P1156R0 rather than the final version; the reader has my apologies for any discrepancies between its claims and the content of the final version of P1156R0.

§ 1.2. What is the preamble for?

It is a near-universal convention that translation units in C++ use `#include` directives to import the interfaces of other libraries, and collect those `#includes` at the start of the translation unit. It is reasonable to consider what benefit we gain by converting this from a mere convention to a rule. There are several technical benefits, including:

- Tools that wish to perform automated management of imports can do so more easily.
- Tools that wish to extract dependencies from source code do not need to process the entire file.
- Collecting the imports into a single block enables compilation strategies where the compiler identifies all the imports first, and then triggers the compilation of all dependencies in parallel.
- In the case of macro importation, all macro imports can be delayed until after the end of the preamble, enabling the set of imports of a translation unit to be determined without having built any of its dependencies.

In addition, promoting this from convention to language rule makes C++ code more uniform.

§ 2. P1156R0 Section 3: Non-Modular Code

One important question that must be answered when determining how a module system will fit into the C++ ecosystem is: how are the dependencies of a translation unit determined?

In traditional compilations of simple programs, each translation unit can be compiled in isolation without an answer to this question. The set of dependencies can be computed as a side-effect of the compilation, and used only to decide when recompilation is necessary.

However, once the need for more complexity arises (for instance, if the program generates header files as part of its build process, or if a distributed build process needs to be sent all the requisite inputs for a compilation), a different strategy is needed. In practice, build systems that tackle such problems either have explicitly-specified dependencies on such headers, or perform `#include` scanning to determine the set of header files used by a source file. (Some build systems combine both strategies, for example requiring a superset of dependencies to be explicitly declared and refined to an actual dependency set via include scanning.)

We will not discuss systems with explicit specification of dependencies further, as dependencies for modules can be explicitly specified in much the same manner as for header files, and with the same benefits and disadvantages. Instead, we focus on `#include` scanning.

There are, broadly-speaking, two different approaches taken to perform `#include` scanning:

- approximate scanning just looks for lines in the source file that appear to be `#include` directives, and assumes the named file is included
- precise scanning performs steps equivalent to a full preprocessing of the source file, with the relevant configuration that will be used for the compilation, and determines the set of files that are actually included

EXAMPLE 1

```
#define F00 "bar.h"
#ifndef F00
#include "baz.h"
#endif
#include F00
```

A precise scan would indicate that this file includes `"bar.h"`, but an approximate scan will likely determine that it includes `"baz.h"`.

Approximate scanning needs no additional language support to work with modules: the scanning tool can assume that the tokens after an `import` are not subject to macro expansion, and that all `import` declarations are reachable, just as it likely does for `#include` directives.

Precise scanning, however, needs to be able to scan (and therefore, in general, preprocess) the entire portion of the file that might contain imports, without already having a precompiled form of those imports. To support this, we require all legacy header imports to precede the point at which imported macros become visible, at least in translation units that have preambles.

Section 3 of P1156R0 suggests that the same rule should also apply to non-module translation units, in order to allow precise dependency scanning to be applied there too. However, this creates a problem. Consider:

```
// a.h
#ifndef A_H
#define A_H
import some.module;
// ...
#endif
```

```
// b.h
#ifndef B_H
#define B_H
#include "x.h"
#include "a.h"
// ...
#endif
```

The author of `"b.h"` may have no idea that `"a.h"` contains a module import -- and nor should they! And yet, after preprocessing, `"b.h"` expands to something like:

```
// ... declarations from x.h ...
import some.module;
// ... declarations from a.h ...
// ... declarations from b.h ...
```

... in which notably the import of `some.module` is in the middle of the translation unit. We cannot disallow these scenarios without blocking reasonable upgrade paths for existing legacy code.

§ 2.1. Proposal

In this paper, we propose a different solution for precise dependency scanning for legacy translation units:

- Disallow the `import header-name ;` syntax outside of a preamble.

That is: within legacy code, we permit import of named modules, but not import of legacy header files. For headers, the traditional `#include` syntax must be used, and it's implementation-defined whether

and when those `#include`s are mapped into imports of legacy header units.

A precise include scanner can then merely collect the named module imports it sees, along with the names of header files that it enters, ambivalent of whether a `#include` will be translated into a legacy header import by the compiler.

A misconfigured build system might result in a non-modular header being treated as a legacy header unit. One consequence of this is that the "precise" include scanner may compute a different result than would be determined by the compiler. For example:

```
// nonmodular.h
#ifdef F00
#define BAR
#endif
```

```
// legacy.cpp
#define F00
#include "nonmodular.h"
#ifndef BAR
#include "other.h"
#endif
```

Here, if the build system is erroneously configured to treat `nonmodular.h` as a legacy header unit, include scanning may determine that `legacy.cpp` does not include `"other.h"`, but when actually compiled, the macro `F00` from `legacy.cpp` will not leak into `nonmodular.h`, and as a result, `other.h` will be included.

However, this is a consequence of a build system misconfiguration, and is just one of many things that will go wrong if non-modular headers are incorrectly configured as modular.

Even in this case, a build system can detect the problem by inspecting a list of dependencies produced by the compiler as a side-effect of compilation (for example using GCC's `-MD` or MSVC's `/showIncludes`).

Such an include scanner would then transitively scan all headers reachable from each visited legacy header, but this is no different from the status quo before modules are introduced. Modular compilations would still benefit from not needing to scan legacy header includes.

§ 3. P1156R0 Section 4: Preamble End

In corner cases where the preamble is immediately followed by the expansion of an imported macro, it is non-trivial to determine where the preamble ends, both for the compiler and for a human reader.

EXAMPLE 2

```
module M;
import "foo.h";
#ifdef F00
int x;
#endif
int y;
```

If `"foo.h"` exports a macro named `F00`, then `x` should be declared according to the rules in P1103R0. But an import *after* the `#ifdef` block changes this behavior:

```
module M;
import "foo.h";
#ifdef F00
int x;
#endif
import bar;
int y;
```

Now the preamble extends to the end of the `import bar;` declaration, and the `#ifdef F00` block is not entered and `x` is not declared.

Various parties have proposed to solve this problem with an explicit preamble end marker. P1156R0 suggests `import;`. The GCC modules implementation suggests using simply `;` (and only in the cases where the end of the preamble is not otherwise obvious). These suggestions introduce verbosity into the preamble that would be better avoided. We discuss some alternatives below.

§ 3.1. Do not allow preprocessor action at the end of the preamble

Perhaps the simplest solution would be to make the corner cases where the end of the preamble is ambiguous ill-formed. More precisely:

- if the *token* following the `;` that ends the preamble in the preprocessed output (in phase 7 of translation) is not formed from the *preprocessing-token* following the `;` in the tokenized input (in

phase 3 of translation), the program is ill-formed

This makes the first example above invalid, because the *preprocessing-token* after the `;` that ends the preamble is the `#` *preprocessing-token* on line 3, but the next *token* in the preprocessed output is the `int` on line 6.

§ 3.2. Defer macro import slightly more

Instead of deferring macro import until just after the last token of the preamble, we could instead defer macro import to just before the first token *after* the preamble.

This simplifies the technical problems with determining the end of the preamble: because it is an error to form the `export` and `import` tokens of an import declaration by macro expansion, we can preprocess the next token after the end of an `import` declaration; if it is neither of those, we can end the preamble, make imported macros visible, and carry on from that token.

Under this approach, the `#ifdef F00` block would not be entered in either of the examples above, because in both cases it is part of the preamble. This does not completely remove the surprise from the example.

§ 3.3. Do not defer macro import

Instead of deferring macro import until the end of the preamble, we could specify:

- macros become visible immediately, but
- if an imported macro is expanded before the end of the preamble, the program is ill-formed

The first rule means that valid code always does what it looks like it does. In the first example above, `F00` is imported and expanded. The second example above is ill-formed, because `F00` was expanded before the end of the preamble.

This also preserves the ability to precisely determine the set of imports of a module unit without having precompiled module artifacts: if the program is valid, then no imported macros can affect the set of imports.

However, this loses the property that module imports can be reordered with no semantic effect on the program: reordering a legacy module unit import may change whether the program is ill-formed (but, if valid, not what it means). Tooling wishing to automatically rearrange imports can still do so by

following the convention that legacy header imports are placed after all other imports (and thus cannot export macros whose names conflict with the names of any imported modules).

This approach also means that a compiler cannot slurp up all the imports first, hand them off to some other process (such as the build system) to compile in parallel, and then resume and process the rest of the file. However, there are good alternatives for an implementation to pursue. For example, the following strategy can be used:

1. while the input stream starts with `import` or `export import` (without to any further preprocessing), preprocess and consume an import directive and add it to our list of imports
2. take the list of imports, import those modules, make their macros visible, and preprocess until you can determine whether the next token(s) are `import` or `export import`; if so, go to step 1
3. we have reached the end of the preamble

(An implementation can check after the fact if any of the identifiers it encountered in this process would have corresponded to an imported macro.)

§ 3.4. Proposal

My preferred option is the final one above: make legacy header imports behave as if they import macros immediately, but make the program ill-formed if an imported macro would have been expanded within the preamble.

This approach provides a reasonable user model: attempts to make the set of imports depend on an imported macro are cleanly rejected rather than being accepted but doing something unexpected.

It also admits a relatively straightforward, single-pass implementation strategy, and allows precise dependency extraction for valid code without the need for pre-built module artifacts for dependencies.

§ 4. P1156R0 Section 5: Module Partitions

We wish to suggest a correction to P1156R0; D1156R0 section 5 says:

The fact that there can (presumably) be both interface and implementation partition units for the same partition further complicates things [...]

However, P1103R0 [dcl.module.unittp3](#) disallows this situation:

A named module shall not contain multiple module partitions with the same sequence of *identifiers* in their *module-partitions*.

That is, the names of module partitions are all distinct, regardless of whether they are interface or implementation partitions. (This must be the case, otherwise the `import :partition;` syntax could not uniquely identify the partition to import.)